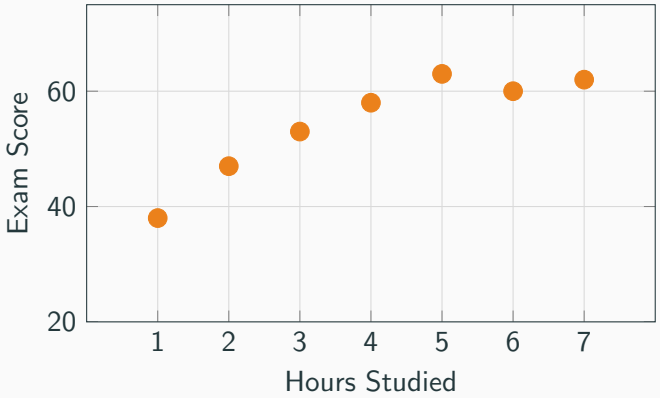
Gradient Descent

Optimization Principles

Hoa T. Vu

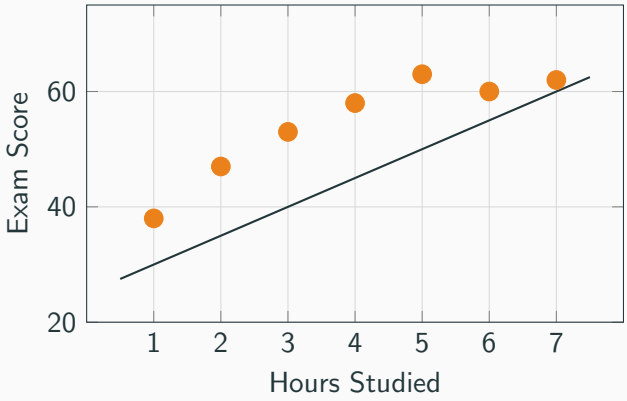
Curve Fitting

Data: Hours Studied vs. Exam Score

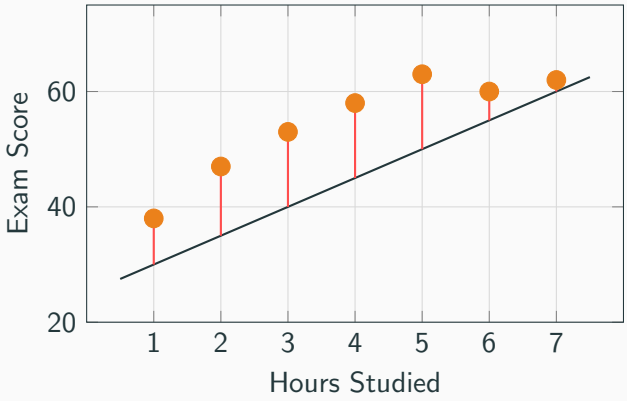


Hours	Score
1	38
2	47
3	53
4	58
5	63
6	60
7	62

What Does the Error Look Like?



What Does the Error Look Like?



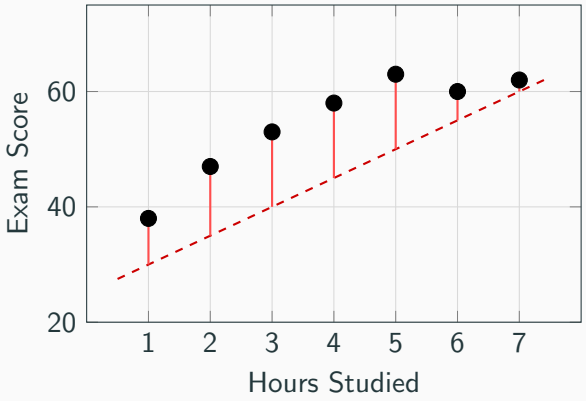
Each red segment is the **residual**:

$$e_i = y_i - \hat{y}_i$$

The loss penalizes large residuals:

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^{n} e_i^2$$

Two Curves, Two MSEs

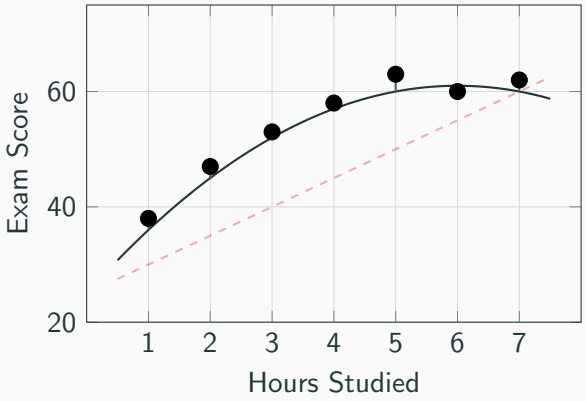


Curve 1 (linear)

$\hat{y} = 25 + 5x$

$\mathcal{L}_1 \approx 106.3$

Two Curves, Two MSEs



Curve 1 (linear)

$\hat{y} = 25 + 5x$

$\mathcal{L}_1 \approx 106.3$

Curve 2 (quadratic)

$\hat{y} = 25 + 12x - x^2$

$\mathcal{L}_2 \approx 3.4$

The Optimization Problem

We want to *minimize* the MSE over the parameter space:

$$\theta^\star = \arg\min_\theta \ \mathcal{L}(\theta)$$

In the previous example, θ represents the coefficients of the curve (e.g., a, b, c for a quadratic $ax^2 + bx + c$).

The Optimization Problem

We want to *minimize* the MSE over the parameter space:

$$\theta^\star = \arg\min_{\theta} \; \mathcal{L}(\theta)$$

In the previous example, θ represents the coefficients of the curve (e.g., a, b, c for a quadratic $ax^2 + bx + c$).

Key Insight
Fitting a curve to data **is** an optimization problem.

Partial Derivatives

The Idea

For $f(x, y)$, the **partial derivative** with respect to x measures how f changes when x moves and y is *held fixed*:

$$\frac{\partial f}{\partial x}(x_0, y_0) = \lim_{h \to 0} \frac{f(x_0 + h,\ y_0) - f(x_0,\ y_0)}{h}$$

The Idea

For $f(x, y)$, the **partial derivative** with respect to x measures how f changes when x moves and y is *held fixed*:

$$\frac{\partial f}{\partial x}(x_0, y_0) = \lim_{h \to 0} \frac{f(x_0 + h,\ y_0) - f(x_0,\ y_0)}{h}$$

In practice: differentiate with respect to x, treating y as a constant.

Symmetrically, $\dfrac{\partial f}{\partial y}$ treats x as a constant.

Geometric Interpretation

$\frac{\partial f}{\partial x}$ is the slope of f along the x-direction:

$$g(x) = f(x,\ y_0) \quad \Rightarrow \quad \frac{\partial f}{\partial x} = g'(x)$$

Examples

Ex 1. $f(x, y) = x^2 + 3xy$

Ex 2. $f(x, y) = e^{xy} + y^2$

Ex 3. $f(x, y, z) = x^2 z + \sin(yz)$

Examples

Ex 1. $f(x, y) = x^2 + 3xy$

$$\frac{\partial f}{\partial x} = 2x + 3y$$
$$\frac{\partial f}{\partial y} = 3x$$

Ex 2. $f(x, y) = e^{xy} + y^2$

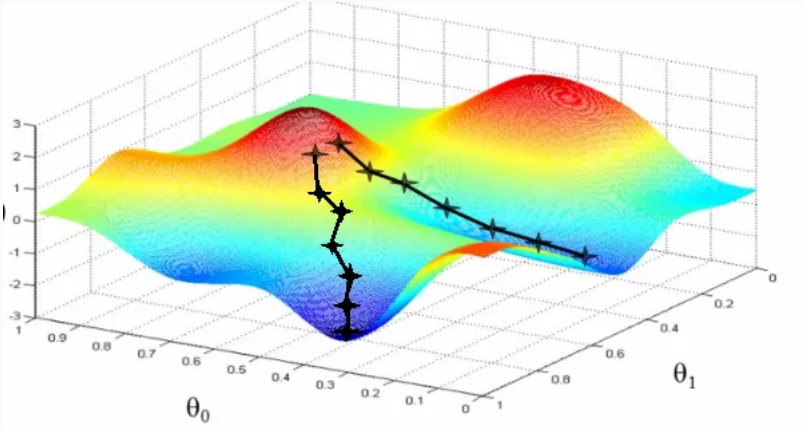
$$\frac{\partial f}{\partial x} = y\, e^{xy}$$
$$\frac{\partial f}{\partial y} = x\, e^{xy} + 2y$$

Ex 3. $f(x, y, z) = x^2 z + \sin(yz)$

$$\frac{\partial f}{\partial x} = 2xz$$
$$\frac{\partial f}{\partial y} = z\cos(yz)$$
$$\frac{\partial f}{\partial z} = x^2 + y\cos(yz)$$

Gradient Descent

Gradient Descent



The Gradient

For $f : \mathbb{R}^n \to \mathbb{R}$, the gradient is the vector of partial derivatives:

$$\nabla f(\theta) = \begin{pmatrix} \frac{\partial f}{\partial \theta_1} \\ \vdots \\ \frac{\partial f}{\partial \theta_n} \end{pmatrix}$$

The Gradient

For $f : \mathbb{R}^n \to \mathbb{R}$, the gradient is the vector of partial derivatives:

$$\nabla f(\theta) = \begin{pmatrix} \dfrac{\partial f}{\partial \theta_1} \\ \vdots \\ \dfrac{\partial f}{\partial \theta_n} \end{pmatrix}$$

It points in the direction of **steepest ascent** at θ.
Gradient descent moves in the **opposite** direction.

Gradient Examples

Example 1. $f(x) = x^2$

$$\nabla f = \frac{df}{dx} = 2x$$

At $x = 3$: gradient $= 6$

Gradient Examples

Example 1. $f(x) = x^2$

$$\nabla f = \frac{df}{dx} = 2x$$

At $x = 3$: gradient $= 6$

Example 2. $f(x, y) = x^2 + y^2$

$$\nabla f = \begin{pmatrix} 2x \\ 2y \end{pmatrix}$$

At $(1, 2)$: gradient $= (2,\ 4)^\top$

Gradient Examples

Example 1. $f(x) = x^2$

$$\nabla f = \frac{df}{dx} = 2x$$

At $x = 3$: gradient $= 6$

Example 2. $f(x, y) = x^2 + y^2$

$$\nabla f = \begin{pmatrix} 2x \\ 2y \end{pmatrix}$$

At $(1, 2)$: gradient $= (2,\ 4)^\top$

Example 3. $f(x, y) = 3x^2 - xy$

$$\nabla f = \begin{pmatrix} 6x - y \\ -x \end{pmatrix}$$

At $(1, 2)$: gradient $= (4,\ -1)^\top$

Exercise: Compute the Gradient

1. $f(x, y) = x^3 + 2x^2y - y^3$

2. $f(x, y) = \ln(x^2 + 1) + e^{xy}$

Exercise: Compute the Gradient

1. $f(x, y) = x^3 + 2x^2y - y^3$

$$\nabla f = \begin{pmatrix} 3x^2 + 4xy \\ 2x^2 - 3y^2 \end{pmatrix}$$

2. $f(x, y) = \ln(x^2 + 1) + e^{xy}$

$$\nabla f = \begin{pmatrix} \frac{2x}{x^2 + 1} + ye^{xy} \\ xe^{xy} \end{pmatrix}$$

Verify by checking that units and signs match at a specific point.

The Update Rule

$$\boxed{\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}(\theta)}$$

$\alpha > 0$ learning rate (step size)

$\nabla_\theta \mathcal{L}$ gradient, direction of steepest *ascent*

Example: $f(x, y) = e^x + (x - 2)\, y$

Partial derivatives:

$$\frac{\partial f}{\partial x} = e^x + y, \qquad \frac{\partial f}{\partial y} = x - 2$$

Example: $f(x, y) = e^x + (x - 2)\, y$

Partial derivatives:

$$\frac{\partial f}{\partial x} = e^x + y, \qquad \frac{\partial f}{\partial y} = x - 2$$

Starting point:	$(x_0,\, y_0) = (0,\, 0)$
Initial value:	$f(0, 0) = e^0 + (0 - 2) \cdot 0 = 1$
Learning rate:	$\alpha = 0.1$

Update 1

Gradient at $(0, 0)$**:**

$$\nabla f\big|_{(0,0)} = \begin{pmatrix} e^0 + 0 \\ 0 - 2 \end{pmatrix} = \begin{pmatrix} 1 \\ -2 \end{pmatrix}$$

Update 1

Gradient at $(0, 0)$**:**

$$\nabla f\big|_{(0,0)} = \begin{pmatrix} e^0 + 0 \\ 0 - 2 \end{pmatrix} = \begin{pmatrix} 1 \\ -2 \end{pmatrix}$$

Step:

$$x_1 = 0 - 0.1 \cdot 1 = -0.1, \qquad y_1 = 0 - 0.1 \cdot (-2) = 0.2$$

Update 1

Gradient at $(0,\ 0)$**:**

$$\nabla f\big|_{(0,0)} = \begin{pmatrix} e^0 + 0 \\ 0 - 2 \end{pmatrix} = \begin{pmatrix} 1 \\ -2 \end{pmatrix}$$

Step:

$$x_1 = 0 - 0.1 \cdot 1 = -0.1, \qquad y_1 = 0 - 0.1 \cdot (-2) = 0.2$$

New value:

$$f(-0.1,\ 0.2) = e^{-0.1} + (-0.1 - 2)(0.2) \approx 0.905 - 0.420 = \mathbf{0.485}$$

Update 1

Gradient at $(0,\ 0)$**:**

$$\nabla f\big|_{(0,0)} = \begin{pmatrix} e^0 + 0 \\ 0 - 2 \end{pmatrix} = \begin{pmatrix} 1 \\ -2 \end{pmatrix}$$

Step:

$$x_1 = 0 - 0.1 \cdot 1 = -0.1, \qquad y_1 = 0 - 0.1 \cdot (-2) = 0.2$$

New value:

$$f(-0.1,\ 0.2) = e^{-0.1} + (-0.1 - 2)(0.2) \approx 0.905 - 0.420 = \mathbf{0.485}$$

$$f_0 = 1 \ \longrightarrow \ f_1 \approx 0.485$$

Update 2

Gradient at $(-0.1,\ 0.2)$**:**

$$\nabla f\big|_{(-0.1,\,0.2)} = \begin{pmatrix} e^{-0.1} + 0.2 \\ -0.1 - 2 \end{pmatrix} \approx \begin{pmatrix} 1.105 \\ -2.1 \end{pmatrix}$$

Update 2

Gradient at $(-0.1,\ 0.2)$**:**

$$\nabla f\big|_{(-0.1,\,0.2)} = \begin{pmatrix} e^{-0.1} + 0.2 \\ -0.1 - 2 \end{pmatrix} \approx \begin{pmatrix} 1.105 \\ -2.1 \end{pmatrix}$$

Step:

$$x_2 = -0.1 - 0.1 \cdot 1.105 = -0.210, \qquad y_2 = 0.2 - 0.1 \cdot (-2.1) = 0.410$$

Update 2

Gradient at $(-0.1,\ 0.2)$**:**

$$\nabla f\big|_{(-0.1,\,0.2)} = \begin{pmatrix} e^{-0.1} + 0.2 \\ -0.1 - 2 \end{pmatrix} \approx \begin{pmatrix} 1.105 \\ -2.1 \end{pmatrix}$$

Step:

$$x_2 = -0.1 - 0.1 \cdot 1.105 = -0.210, \qquad y_2 = 0.2 - 0.1 \cdot (-2.1) = 0.410$$

New value:

$$f(-0.210,\ 0.410) = e^{-0.210} + (-2.210)(0.410) \approx 0.810 - 0.906 = \mathbf{-0.096}$$

Update 2

Gradient at $(-0.1,\ 0.2)$**:**

$$\nabla f\big|_{(-0.1,\,0.2)} = \begin{pmatrix} e^{-0.1} + 0.2 \\ -0.1 - 2 \end{pmatrix} \approx \begin{pmatrix} 1.105 \\ -2.1 \end{pmatrix}$$

Step:

$$x_2 = -0.1 - 0.1 \cdot 1.105 = -0.210, \qquad y_2 = 0.2 - 0.1 \cdot (-2.1) = 0.410$$

New value:

$$f(-0.210,\ 0.410) = e^{-0.210} + (-2.210)(0.410) \approx 0.810 - 0.906 = \mathbf{-0.096}$$

$$1 \longrightarrow 0.485 \longrightarrow -0.096 \quad \text{(function is decreasing)}$$

Pseudo-code

```
initialize theta
set learning rate alpha

repeat until convergence:
    g = gradient of L w.r.t. theta
    theta = theta - alpha * g
```

Gradient for Quadratic Fitting

Fitting $\hat{y} = ax^2 + bx + c, \quad \theta = (a, b, c)$

```
y_hat = a*X**2 + b*X + c
err   = y_hat - y            # residuals

grad_a = 2 * (err * X**2).mean()
grad_b = 2 * (err * X).mean()
grad_c = 2 * err.mean()

a -= alpha * grad_a
b -= alpha * grad_b
c -= alpha * grad_c
```

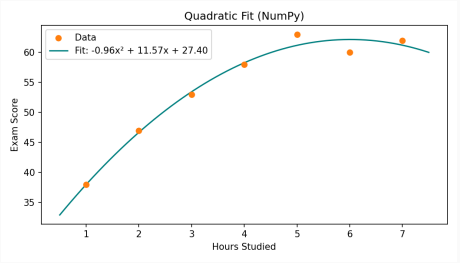
Python (NumPy)

```python
import numpy as np

X = np.array([1,2,3,4,5,6,7], dtype=float)
y = np.array([42,48,59,61,70,73,80], dtype=float)

theta = np.zeros(3)   # [a, b, c]
alpha = 1e-3

for _ in range(50_000):
    y_hat = theta[0]*X**2 + theta[1]*X + theta[2]
    err   = y_hat - y
    grad  = np.array([
        2*(err*X**2).mean(),
        2*(err*X).mean(),
        2*err.mean()])
    theta -= alpha * grad
```



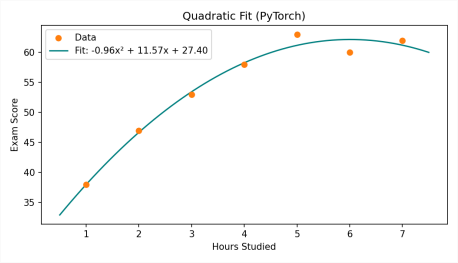
PyTorch

```
import torch

X = torch.tensor([1,2,3,4,5,6,7],
                 dtype=torch.float)
y = torch.tensor([42,48,59,61,70,73,80],
                 dtype=torch.float)

theta = torch.zeros(3, requires_grad=True)
opt   = torch.optim.SGD([theta], lr=1e-3)

for _ in range(50_000):
    y_hat = theta[0]*X**2 + theta[1]*X + theta[2]
    loss  = ((y - y_hat)**2).mean()
    opt.zero_grad()
    loss.backward()
    opt.step()
```



Learning Rate Scheduling

Why Schedule the Learning Rate?

A fixed α is a compromise:

Too large fast early progress, but overshoots near the minimum

Too small stable convergence, but very slow

Schedule start large, decay over time — get both benefits

Why Schedule the Learning Rate?

A fixed α is a compromise:

Too large fast early progress, but overshoots near the minimum

Too small stable convergence, but very slow

Schedule start large, decay over time — get both benefits

$$\alpha_t = \text{schedule}(\alpha_0,\ t)$$

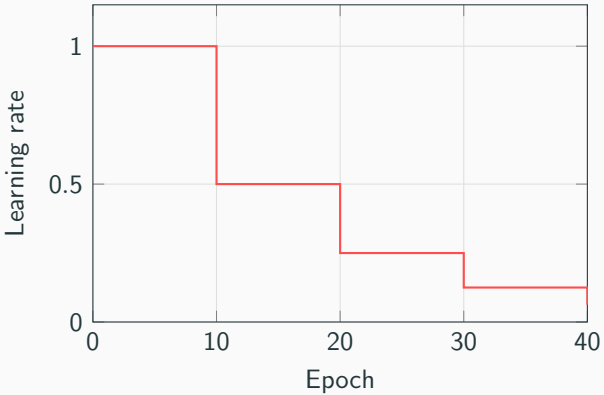
Step Decay

Multiply α by $\gamma < 1$ every k epochs:

$$\alpha_t = \alpha_0 \cdot \gamma^{\lfloor t/k \rfloor}$$

α_0	initial rate
γ	decay factor (e.g. 0.5)
k	step size (e.g. 10)

Large drops at fixed intervals.
Simple, widely used.



Exponential Decay

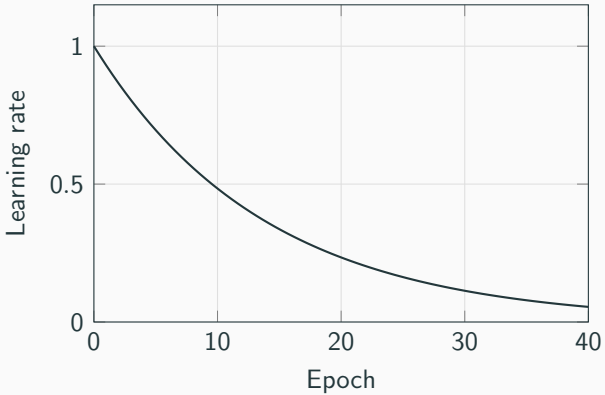
Multiply α by γ after every epoch:

$$\alpha_t = \alpha_0 \cdot \gamma^t$$

- α_0 initial rate
- γ decay factor (e.g. 0.93)

Smooth, continuous decay.

Rate never reaches zero.



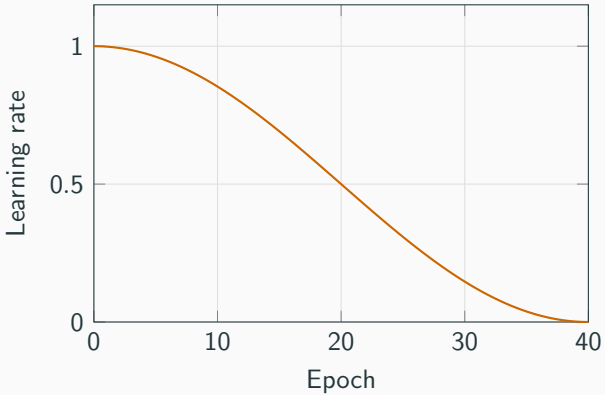
Cosine Annealing

Follow a half-cosine curve over T epochs:

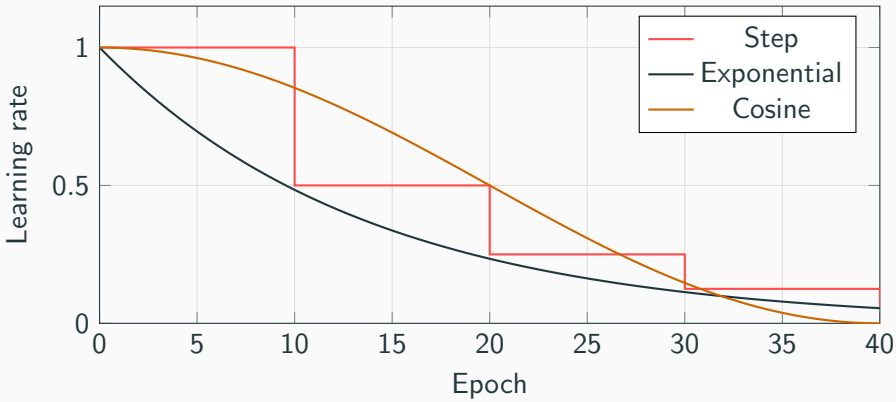
$$\alpha_t = \frac{\alpha_0}{2}\left(1 + \cos\frac{\pi t}{T}\right)$$

α_0	initial rate
T	total epochs

Slow start, fast middle, soft landing.
Very popular in deep learning.



Comparison


Step
Exponential
Cosine
Learning rate
Epoch
0
0.5
1
0
5
10
15
20
25
30
35
40

PyTorch Schedulers

Setup

```
opt = torch.optim.SGD([theta], lr=0.1)

# pick one:
sched = torch.optim.lr_scheduler.StepLR(
    opt, step_size=10, gamma=0.5)

sched = torch.optim.lr_scheduler.ExponentialLR(
    opt, gamma=0.93)

sched =
     torch.optim.lr_scheduler.CosineAnnealingLR(
    opt, T_max=100)
```

Training loop

```
for epoch in range(num_epochs):
    # forward pass
    y_hat = theta[0]*X**2 + theta[1]*X + theta[2]
    loss  = ((y - y_hat)**2).mean()

    # backward pass
    opt.zero_grad()
    loss.backward()
    opt.step()

    # update learning rate
    sched.step()
```

Convergence Conditions (Robbins–Monro)

For gradient descent to converge, $\{\alpha_t\}$ must satisfy:

1. $\sum_{t=0}^{\infty} \alpha_t = \infty$ steps large enough to reach any point

Convergence Conditions (Robbins–Monro)

For gradient descent to converge, $\{\alpha_t\}$ must satisfy:

1. $\sum_{t=0}^{\infty} \alpha_t = \infty$ — steps large enough to reach any point
2. $\sum_{t=0}^{\infty} \alpha_t^2 < \infty$ — steps shrink fast enough to suppress noise

Convergence Conditions (Robbins–Monro)

For gradient descent to converge, $\{\alpha_t\}$ must satisfy:

1. $\sum_{t=0}^{\infty} \alpha_t = \infty$ — steps large enough to reach any point
2. $\sum_{t=0}^{\infty} \alpha_t^2 < \infty$ — steps shrink fast enough to suppress noise

Example ($c = 1$, $p = 1$, harmonic): $\alpha_t = \frac{1}{t}$, $\sum \frac{1}{t} = \infty$ (cond. 1) $\sum \frac{1}{t^2} = \frac{\pi^2}{6} < \infty$ (cond. 2)

Convergence Conditions (Robbins–Monro)

For gradient descent to converge, $\{\alpha_t\}$ must satisfy:

1. $\sum_{t=0}^{\infty} \alpha_t = \infty$ — steps large enough to reach any point
2. $\sum_{t=0}^{\infty} \alpha_t^2 < \infty$ — steps shrink fast enough to suppress noise

Example ($c = 1$, $p = 1$, harmonic): $\alpha_t = \frac{1}{t}$, $\sum \frac{1}{t} = \infty$ (cond. 1) $\sum \frac{1}{t^2} = \frac{\pi^2}{6} < \infty$ (cond. 2)

Fixed learning rate
Satisfies condition 1, but violates condition 2. Converges only to a neighborhood of the minimum.

Exercise: Does the Schedule Converge?

Recall: need $\sum \alpha_t = \infty$ and $\sum \alpha_t^2 < \infty$.

1. $\alpha_t = \dfrac{1}{t^2}$

2. $\alpha_t = \dfrac{1}{t^{3/4}}$

Exercise: Does the Schedule Converge?

Recall: need $\sum \alpha_t = \infty$ and $\sum \alpha_t^2 < \infty$.

1. $\alpha_t = \dfrac{1}{t^2}$

$$\sum \frac{1}{t^2} = \frac{\pi^2}{6} < \infty$$

Condition 1 is **violated**. Does not converge.

2. $\alpha_t = \dfrac{1}{t^{3/4}}$

$$\sum \frac{1}{t^{3/4}} = \infty \quad \text{(cond. 1)}$$

$$\sum \frac{1}{t^{3/2}} < \infty \quad \text{(cond. 2)}$$

Both satisfied. Converges.